

Contents

Introduction	1
Basics	1
Vim config	1
C++ Boilerplate	1
Reading input values	1
Binary	1
Useful Data Types	1
Defining Types	1
Number Types	1
Bitsets	2

Introduction

This document contains commonly used code for competitive programming in c++ as preparation for the 2026 nordic-baltic olympiad of informatics.

Basics

Vim config

Here is a basic vim config for competitive programming in rust:

```
let mapleader = " "
nnoremap <leader>ff :%!rustfmt<cr>
nnoremap <leader>fm :Explore<cr>
tnoremap <Esc> <C-\><C-n>

syntax on
set clipboard=unnamedplus
set mouse=a
set hlsearch
set incsearch
set ignorecase
set smartcase

set number
set relativenumber
set cursorline
set nowrap

set tabstop=4
set shiftwidth=4
set expandtab
set autoindent
set smartindent

hi CursorLine cterm=NONE ctermbg=236
hi MatchParen ctermfg=0 ctermbg=188
```

Place this in `~/.vimrc` or `~/.config/nvim/init.vim` at the start of the competition.

C++ Boilerplate

```
#include <iostream>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
}
```

Note

Some environments provide `#include <bits/stdc++.h>`, but it is not always present. This non-standard header file imports all the standard libraries and avoids having to import many things in the same file.

Reading input values

Reading a single value:

```
int n;
cin >> n;
```

Reading multiple values on the same line. Types can also be mixed in the same line, for example

```
int u, v;
Permission k;
cin >> u >> v >> k;
```

Reading values from multiple lines:

```
vector<int> xs(n);
for (int i = 0; i < n; i++) {
    int x;
    cin >> x;
    xs[i] = x;
}
```

Reading n values from the same line: TODO

Binary

Parsing a binary value:

```
#include <string>

string input;
cin >> input;
int x = stoi(input, nullptr, 2);
```

Note

The radix 2 can also be replaced with any other integer.

Alternatively, a bitset can be used, as described in that section of this document.

Useful Data Types

Defining Types

New types can be defined with `using` as follows

```
using ll = long long;
```

Number Types

Signed integers:

Type	Bits
char	8
short	16
int	16 / 32
long	32
long long	32

Note

These can be prefixed with `unsigned` to use the unsigned variants.

Floating-point numbers:

- `float` - 32 bits, accurate to about 7 decimal digits.
- `double` - 64 bits, accurate to about 15-17 decimal digits.
- `long double` - 80 or 128 bits.

For competitive programming, it is often better to use `<cstdint>` to provide the exact desired size.

Type	Bits
<code>int8_t</code>	8
<code>int16_t</code>	16
<code>int32_t</code>	32
<code>int64_t</code>	32
<code>uint8_t</code>	8
<code>uint16_t</code>	16
<code>uint32_t</code>	32
<code>uint64_t</code>	32

Bitsets

Bitsets are essentially boolean arrays but optimized to use only a single bit per value and allowing the use of boolean operators. It is for example useful when dealing with bitmasks. They provide the following operations

Operator	Operation
<code>&</code>	AND
<code> </code>	OR
<code>^</code>	XOR
<code>~</code>	NOT
<code><<</code>	Left Shift
<code>>></code>	Right Shift

And here are some useful functions:

Operator	Operation
<code>b.count()</code>	Return the number of <code>1</code> bits
<code>b.any()</code>	True if at least one bit is <code>1</code>
<code>b.all()</code>	True if all bits are <code>1</code>
<code>b.none()</code>	True if no bits are <code>1</code>
<code>b.size()</code>	Return the total number of bits
<code>b.to_string()</code>	Convert to a string of binary
<code>b.to_ulong()</code>	Convert to unsigned long
<code>b.to_ullong()</code>	Convert to unsigned long long
<code>b.set(i)</code>	Set the bit at <code>i</code> to <code>1</code>
<code>b.reset(i)</code>	Set the bit at <code>i</code> to <code>0</code>
<code>b.flip(i)</code>	Flip the bit at <code>i</code>
<code>b[i]</code>	Access the bit at index <code>i</code>
<code>b.test(i)</code>	Access bit (with bounds check)

```
#include <bitset>

bitset<10> b; // 0000000000
b.set(2); // 0000000100
b[2]; // true
b[1]; // false
```

Here is an example from my solution to

`nio23-finale-noekkelkort`.

```
#include <bitset>
#include <iostream>
#include <queue>
#include <tuple>
#include <vector>
using namespace std;

using Permission = bitset<30>;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, k, t;
    cin >> n >> m >> k >> t;

    int j = 0;
    vector<tuple<int, int, Permission>> connections(t);
    vector<vector<pair<int, Permission>>> graph(n);
    for (int i = 0; i < m; i++) {
        int u, v;
        Permission k;
        cin >> u >> v >> k;

        graph[u].push_back({v, k});
        graph[v].push_back({u, k});

        if (j < t) {
            connections[i] = {u, v, k};
        }
        j++;
    }

    // O(t (n + m))
    for (const auto &[u, v, k] : connections) {
        deque<int> queue;
        vector<bool> visited(n);
        Permission permission = ~k;
        queue.push_back(u);
        visited[u] = true;

        while (!queue.empty()) {
            int a = queue.front();
            queue.pop_front();

            if (a == v) {
                cout << "0" << endl;
                goto valid;
            }

            for (const auto &[b, k] : graph[a]) {
                if (!visited[b] && (permission & k).any()) {
                    queue.push_back(b);
                    visited[b] = true;
                }
            }
        }

        cout << "1" << endl;
    }
    valid:;
}
```